

Credit Name: CSE 3120

Assignment Name: Chapter 7

Question #1

The encapsulated members of the Circle class, are:

- private double radius
- private static final double PI = 3.14;

Question #2

Same name as the class

Eg:

```
public class example {  
    public example() {  
        // Initialize code  
    }  
}
```

Question #3

- **private** - Can only be seen by things inside the class
- **public** - Visible to anything inside the project

Question #4

dot.radius = 5; does not work, because .radius() is not one of the methods in Circle should bet .setRadius(5) instead

Question #5

- a) The class is named Roo
- b) The data member is named x
- c) getX()
- d) setX()
- e) The helper method is factor()
- f) The constructor is named Roo
- g) There are 4 method members:
 - i) setX()
 - ii) getX()
 - iii) calculate()
 - iv) factor()

Question #6

- **class** - sort of like a template/blueprint, it serves the purpose of defining the variables that will be needed or used but it does not do anything with them on its own
- **object** - the object is what defines the variables that makeup a class
- An example of the two is when buying icecream, the structure/general idea of the ice cream is the class ie: the flavour, number of scoops, and whether it has a cone or not,

but the object is the actual values that makeup the icecream ie: maple walnut, 1 scoop, with a cone

Question #9

- a) z is the constant because of the use of "final"
- b) All three variables are data members because they're in the class not a method
- c) private double y is the instance variable because it does not have "static" in it
- d) All three are class members because they're in the class itself

Question #11

- **overloading method** - occurs in the same class when one method has the same name as another method
- **overriding method** - occurs in separate classes, and replaces one of the methods eg: you had a Manager class with the methods: getPay(), and payEmployees along with an Employee class with the method getPay(), if you override the managers getPay() with the employees, then the getPay() method will change but the payEmployees() method won't